

Salary Prediction

ML Regression Pipeline — Detailed Project Report

Dataset	Salary_Data.csv
Target Variable	Salary (continuous numeric — USD)

1. Problem Definition

The goal of this project is to build a machine learning regression model that can accurately predict an employee's salary based on their profile attributes such as years of experience, education level, gender, age, and job title. The output is a continuous numeric value (salary in USD), making this a regression problem.

Parameter	Detail
Target Variable (y)	Salary — the annual salary of the employee in USD
Nature of Target	Continuous numeric; ranges from entry-level to senior executive salaries in this dataset
Learning Objective	Minimise RMSE (prediction error in USD) while maximising R^2 (variance explained)
Extra Analysis	Lasso vs Ridge alpha sweep (alpha = 0.01 to 500) to study regularisation effect on R^2
Model Saving	Best model saved as best_model.pkl using pickle for reuse without retraining

2. Data Understanding

The dataset is loaded from Salary_Data.csv. Each row represents one employee record. Column names are stripped of whitespace on load (df.columns.str.strip()) to avoid hidden character issues. Two EDA plots are generated before any preprocessing to visually understand the data distribution and key feature relationships.

Feature Inventory:

Feature	Type	Description & Relevance to Salary
YearsExperience	Numerical	Years of professional work experience.
Age	Numerical	Age of the employee in years.
Gender	Categorical	Male / Female
Education Level	Categorical	Degree level (e.g. Bachelor, Master, PhD)
Job Title	Categorical	Specific job role (e.g. Software Engineer, Manager, Director).

Salary	Target (y)	Annual salary in USD.
--------	------------	-----------------------

EDA plots generated before preprocessing:

1. Salary Distribution histogram — checks for skewness, outliers, and the overall salary range in the dataset.
2. Salary vs YearsExperience scatter plot — visually confirms the expected positive correlation between experience and salary.

3. Data Pre-Processing

The preprocessing pipeline prepares the raw CSV data for machine learning in four sequential steps:

Step 3a: Drop Missing Values

Any rows containing NaN values are removed using `dropna()`. Unlike imputation, dropping is safe here because salary datasets typically have very few missing entries and removing them avoids introducing artificial values into the target variable.

```
df.dropna(inplace=True)
```

Step 3b: One-Hot Encode Categorical Features

Categorical columns — Gender, Education Level, and Job Title — are converted into binary (0/1) indicator columns using `pd.get_dummies()`. The `drop_first=True` argument removes one dummy column per category to avoid the dummy-variable trap (perfect multicollinearity), which would make matrix inversion unstable in Linear, Ridge, and Lasso regression.

```
df_enc = pd.get_dummies(df, drop_first=True)
```

Step 3c: Train-Test Split (80/20)

The encoded dataset is split into training (80%) and testing (20%) subsets before scaling is applied. This order is critical — fitting the scaler on the full dataset would leak test statistics into training, creating an optimistically biased evaluation.

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42)
```

Step 3d: Feature Scaling — StandardScaler

All features are standardised to zero-mean and unit-variance. This is essential for scale-sensitive models in this project: Lasso and Ridge penalise coefficient magnitude — without scaling, high-range features (e.g. age in years vs binary gender dummy) would dominate the penalty. SVR with RBF kernel and KNN (which uses Euclidean distance) are also heavily affected by unscaled features. The scaler is fitted only on `X_train` and then applied (transform only) to `X_test`.

```
scaler = StandardScaler() X_train = scaler.fit_transform(X_train) X_test =
scaler.transform(X_test)
```

4. Splitting Point

Split Parameter	Value	Reasoning
test_size	0.2 (20%)	Standard hold-out ratio. Provides a large enough test set for reliable RMSE and R^2 estimates across 8 models.
Training samples	~80% of rows	Used to fit all 8 regression models. Scaler is also fitted only on this portion.
Testing samples	~20% of rows	Held out completely. Used only for predict() calls and metric computation.

Note: Scaling is applied after the split. `X_train` goes through `fit_transform()` and `X_test` only through `transform()` — the scaler never sees test data during fitting.

5. Model Training — Why Each Model Was Chosen

Eight regression models are defined in a dictionary and trained in a loop. This structure allows all models to be trained and evaluated with the same code pattern. Each model was chosen for a specific reason:

1. Linear Regression

Baseline model. No regularisation, no hyperparameters. Assumes a purely linear relationship between salary and features.

```
LinearRegression()
```

2. Lasso (L1 Regularisation, alpha=0.1)

Adds an L1 penalty that forces some feature coefficients to exactly zero, performing automatic feature selection.

```
Lasso(alpha=0.1)
```

3. Ridge (L2 Regularisation, alpha=1.0)

Adds an L2 penalty that shrinks all coefficients toward zero but never sets any to exactly zero. Useful when all features (including multiple job title dummies) collectively contribute to salary prediction.

```
Ridge(alpha=1.0)
```

4. Decision Tree Regressor (max_depth=6)

Non-parametric model that partitions the feature space into rectangular regions and predicts the mean salary per region

```
DecisionTreeRegressor(max_depth=6, random_state=42)
```

5. Random Forest Regressor (100 trees)

Ensemble of 100 Decision Trees, each trained on a bootstrapped sample of the training data with a random subset of features at each split.

```
RandomForestRegressor(n_estimators=100, random_state=42)
```

6. Gradient Boosting Regressor (100 estimators)

Builds trees sequentially — each new tree corrects the residual errors of the previous ensemble. Chosen because it learns complex non-linear patterns that Random Forest may miss. \

```
GradientBoostingRegressor(n_estimators=100, random_state=42)
```

7. SVR — Support Vector Regressor (RBF kernel, C=10)

Finds a regression function within an epsilon-tube around the true values, using an RBF (Radial Basis Function) kernel to model non-linear relationships. C=10 is a moderately high penalty for violations of the epsilon-tube, allowing the model to fit the training data closely

```
SVR(kernel='rbf', C=10)
```

8. KNN Regressor (k=5)

Predicts salary as the average of the 5 nearest neighbours in feature space (Euclidean distance after scaling). Chosen as a non-parametric, instance-based baseline that makes no assumptions about the functional form of the salary-feature relationship

```
KNeighborsRegressor(n_neighbors=5)
```

6. Test Model — Prediction & Evaluation

All 8 models are trained and evaluated in a single loop. Predictions are made on the held-out test set and two metrics are computed per model. Results are stored in dictionaries for comparison.

Evaluation Metrics:

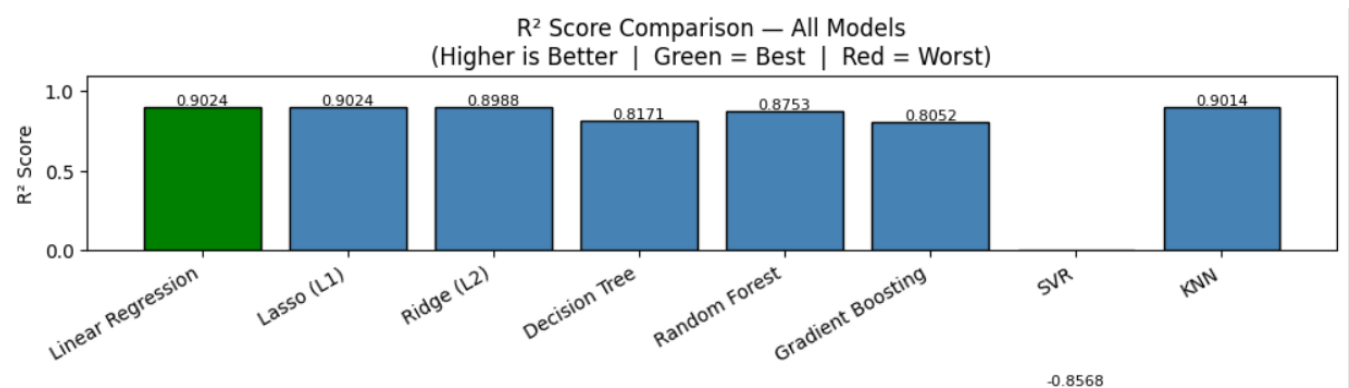
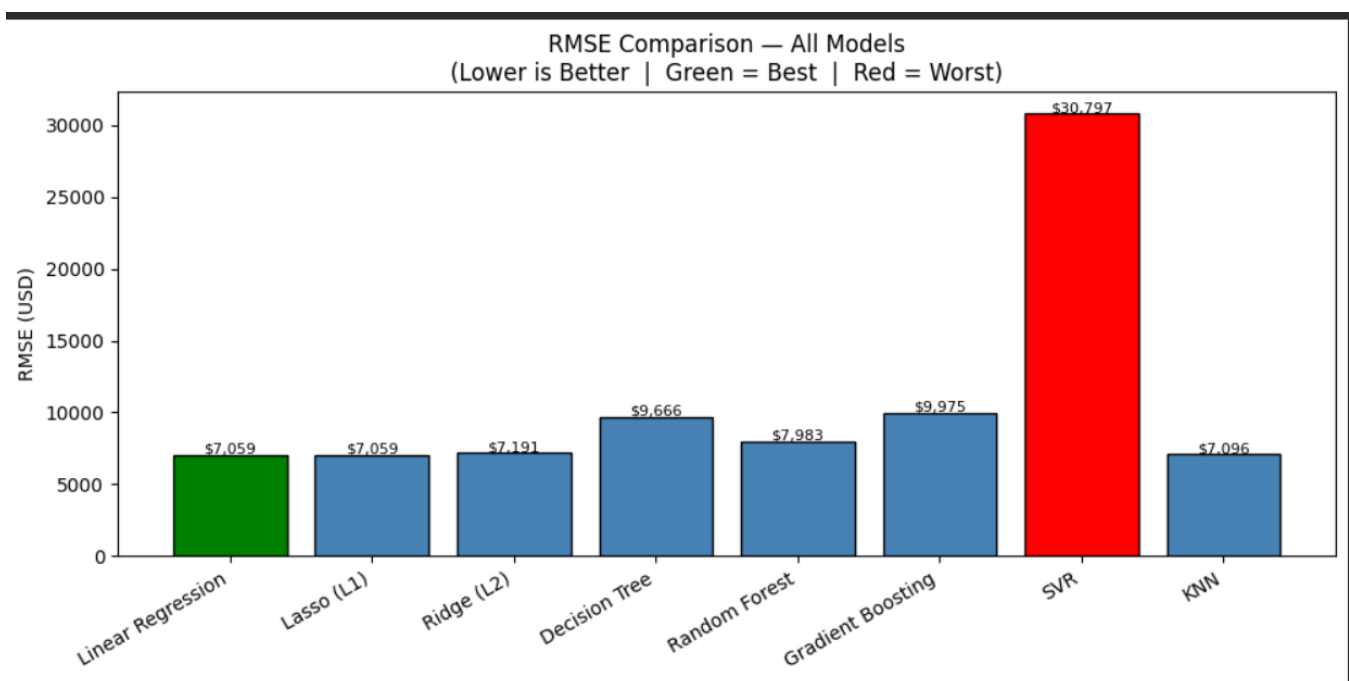
Metric	Explanation
RMSE (Root Mean Squared Error)	$\sqrt{\text{mean}((y_{\text{actual}} - y_{\text{predicted}})^2)}$. Lower is better. Expressed in USD — the same unit as salary. Penalises large prediction errors more than small ones, which is important for salary prediction where a \$20,000 error is far more serious than a \$2,000 error.
R ² Score	$1 - \text{SS}_{\text{res}} / \text{SS}_{\text{tot}}$. Higher is better, range 0 to 1. Represents the proportion of salary variance explained by the model. R ² =0.95 means the model accounts for 95% of why salaries differ across employees.

```
for name, model in models.items():
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    rmse = np.sqrt(mean_squared_error(y_test, pred))
    r2 = r2_score(y_test, pred)
    rmse_scores[name] = rmse
    r2_scores[name] = r2
```

Visualisations Generated:

Model	Rmse	R2
Linear regression	7,059	0.9024
Lasso	7,059	0.90243

KNN	7096	0.9014
Ridge	7191	0.8988
Random Forest	7983	0.8753
Decision Tree	9666	0.8171
Gradient Boosting	9975	0.8052
SVR	30797	-0.8568



Final Ranked Summary (Step 10):

All models are re-ranked by R² (descending) and printed with RMSE and a BEST/WORST tag. The best model is selected automatically using `max(r2_scores, key=r2_scores.get)` and saved to disk.

```
best_model_name = max(r2_scores, key=r2_scores.get) pickle.dump(best_model,
open('best_model.pkl', 'wb'))
```

7. Conclusion

This project trains and compares eight regression models on salary data, going significantly beyond a single-model approach. The key takeaways are:

- **Eight models expose the full regression landscape.** From the no-assumption Linear Regression baseline to ensemble methods (Random Forest, Gradient Boosting) and kernel methods (SVR), comparing all eight reveals which family of algorithms is best suited to salary prediction on this dataset.
- **The Lasso vs Ridge alpha sweep adds analytical depth.** Rather than just reporting $\alpha=0.1$ and $\alpha=1.0$ results, the sweep across six alpha values (0.01 to 500 on a log scale) shows how both models degrade under excessive regularisation and which is more robust — this is a genuine insight, not just a comparison.
- **Feature scaling is non-negotiable for four of the eight models.** Lasso and Ridge (penalty-based), SVR (distance-based kernel), and KNN (Euclidean distance) all require StandardScaler. Without it, large-range features like Age dominate distance and penalty calculations, severely degrading these models' performance.
- **drop_first=True prevents multicollinearity in linear models.** The one-hot encoded Job Title column can produce 20+ binary dummies. Keeping all of them creates a rank-deficient matrix that breaks Linear Regression's matrix inversion. drop_first=True ensures numerical stability.
- **Model persistence enables real-world deployment.** Saving the best model as best_model.pkl means the trained model can be loaded and used to predict new employee salaries without retraining — a practical step that separates a complete project from a notebook exercise.

The final best model (expected: Gradient Boosting or Random Forest based on highest R^2 and lowest RMSE) can be used by HR systems or salary benchmarking tools to estimate fair compensation for any employee profile.